

MongoDB Aggregation Pipelines

Questions, MongoDB Commands, and Sample Outputs

1. Find orders placed in the last 30 days using \$match

This pipeline dynamically finds orders from the previous 30 days using \$\$NOW and \$dateSubtract.

```
db.orders.aggregate([
  {
    $match: {
      orderDate: {
        $gte: {
          $dateSubtract: {
            startDate: "$$NOW",
            unit: "day",
            amount: 30
          }
        },
        $lte: "$$NOW"
      }
    }
  }
])
```

Sample Output

```
[
  {
    _id: 1,
    userId: "U101",
    items: [
      { product: "Laptop", quantity: 1, price: 55000 }
    ],
    orderDate: ISODate("2026-09-05T10:30:00Z")
  },
  {
    _id: 2,
    userId: "U102",
    items: [
      { product: "Headphones", quantity: 2, price: 2500 }
    ],
    orderDate: ISODate("2026-08-28T14:20:00Z")
  }
]
```

2. Calculate total amount spent by each user using \$group

The \$unwind stage processes each item, while \$multiply calculates quantity × price. \$group then totals the amount for each user.

```
db.orders.aggregate([
  {
    $unwind: "$items"
  },
  {
    $group: {
      _id: "$userId",
      totalAmountSpent: {
        $sum: {
          $multiply: [
            "$items.quantity",
            "$items.price"
          ]
        }
      }
    }
  },
  {
    $project: {
```

```
        _id: 0,  
        userId: "$_id",  
        totalAmountSpent: 1  
    }  
}  
])
```

Sample Output

```
[  
  { userId: "U101", totalAmountSpent: 65000 },  
  { userId: "U102", totalAmountSpent: 12500 },  
  { userId: "U103", totalAmountSpent: 8400 }  
]
```

3. Find the top 3 highest-rated cafes in Ahmedabad

The pipeline filters for cafes in Ahmedabad, sorts by rating in descending order, and limits the result to three restaurants.

```
db.restaurants.aggregate([
  {
    $match: {
      cuisine: "Cafe",
      city: "Ahmedabad"
    }
  },
  {
    $sort: {
      rating: -1
    }
  },
  {
    $limit: 3
  }
])
```

Sample Output

```
[
  {
    _id: 5,
    name: "The Green Cafe",
    cuisine: "Cafe",
    city: "Ahmedabad",
    rating: 4.8
  },
  {
    _id: 2,
    name: "Cafe Mocha",
    cuisine: "Cafe",
    city: "Ahmedabad",
    rating: 4.7
  },
  {
    _id: 8,
    name: "Urban Brew",
    cuisine: "Cafe",
    city: "Ahmedabad",
    rating: 4.6
  }
]
```

4. Calculate total box office collection for each genre

The \$group stage totals boxOffice for each genre. The final \$project removes the default _id field as required.

```
db.movies.aggregate([
  {
    $group: {
      _id: "$genre",
      totalCollection: {
        $sum: "$boxOffice"
      }
    }
  },
  {
    $project: {
      _id: 0,
      genre: "$_id",
      totalCollection: 1
    }
  }
])
```

Sample Output

```
[
  { genre: "Action", totalCollection: 1850000000 },
  { genre: "Comedy", totalCollection: 920000000 },
  { genre: "Drama", totalCollection: 750000000 }
]
```

5. Daily sales report for the last week

This pipeline filters payments from the last 7 days, groups them by calendar day using \$dateTrunc, calculates total sales with \$sum, and sorts the report chronologically.

```
db.payments.aggregate([
  {
    $match: {
      paymentDate: {
        $gte: {
          $dateSubtract: {
            startDate: "$$NOW",
            unit: "day",
            amount: 7
          }
        },
        $lte: "$$NOW"
      }
    },
    {
      $group: {
        _id: {
          $dateTrunc: {
            date: "$paymentDate",
            unit: "day"
          }
        },
        totalSales: {
          $sum: "$amount"
        }
      }
    },
    {
      $project: {
        _id: 0,
        date: "$_id",
        totalSales: 1
      }
    },
    {
      $sort: {
        date: 1
      }
    }
  ])
```

Sample Output

```
[
  { date: ISODate("2026-09-05T00:00:00Z"), totalSales: 18500 },
  { date: ISODate("2026-09-06T00:00:00Z"), totalSales: 24300 },
  { date: ISODate("2026-09-07T00:00:00Z"), totalSales: 19800 },
  { date: ISODate("2026-09-08T00:00:00Z"), totalSales: 32100 },
  { date: ISODate("2026-09-09T00:00:00Z"), totalSales: 27500 },
  { date: ISODate("2026-09-10T00:00:00Z"), totalSales: 29400 },
  { date: ISODate("2026-09-11T00:00:00Z"), totalSales: 35200 }
]
```

Changes made to fit the payments collection

The pipeline assumes the collection contains userId, amount, and paymentDate. paymentDate is used for the date filter and daily grouping, while amount is summed to calculate daily sales. The sample output is illustrative; actual results depend on the documents stored in MongoDB.